

Verilator as a UVM host

Browser sketches taught UVM methodology, phases, factory, agents, sequences

Host model: compile-to-C++ vs event-driven

- Classic SystemVerilog simulators are event-driven
- Verilator is a compile-to-C++ flow
- UVM still runs as SystemVerilog when the build includes uvm_pkg and your test
- That is why lint, compile time
- Tool depth for Verilator alone lives in learn Verilator

Practical limits — what to expect

- Expect longer first compiles when uvm_pkg joins the Verilator build
- Expect to respect Verilator's supported SystemVerilog subset
- Expect cycle-oriented stepping
- Expect Makefile and UVM home discipline: unset paths fail before meaningful compile output
- Do **not** expect this bridge to replace a site's full UVM sign-off flow

Related literacy (optional)

- No browser lab is required for this bridge
- Run stages, and when teams pick Icarus versus Verilator for which job
- Those tools do not compile Accellera UVM, they sharpen the host mental model
- Then return here and continue to module twenty-three for UVM home, include paths

Pitfalls to watch

- Do not treat Verilator as “just a faster VCS”, the host model differs
- This one is methodology on Verilator
- Do not assume every UVM example from a commercial flow ports unchanged

Your turn

- Complete the bridge checklist
- Optional: skim sim-pipeline or iverilog-versus-Verilator for stage vocabulary
- Module twenty-four for the actual Verilator UVM run